

2026-02-10

Sometimes, You Aren't What You Do: Mimicry Attacks against Provenance Graph Host Intrusion Detection Systems

Evading Provenance-Based ML Detectors with Adversarial System Actions

质疑一：有效性

现有对抗样本方法往恶意样本里加良性内容来骗检测器。但加的东西在真实攻击场景中会有副作用，可能破坏攻击本身。

Android场景下这个问题被回避了（不执行/无效果/组件隔离）。但真实攻击场景中，加的良性行为必须真执行，执行了就有真实副作用。

保证攻击有效性（功能保持）的安卓论文

所有论文都声称保证了，但机制完全不同

安卓/PE对抗样本中功能保持的方法，能参考到溯源图场景的

BagAmmo (USENIX'23) — 图效果大、系统效果小

▼ Plain Text | 复制代码

```
1  安卓怎么做：
2    try-catch trap，注入的代码不执行
3    静态分析看得到（FCG多了边）
4    运行时走不到（零副作用）
5
6  参考什么：
7    让检测器看到你想让它看到的，而真实系统影响极小
8
9  溯源图怎么用：
10   寻找read-only或低副作用操作
11   比如read一个不相关文件 → provenance graph多了一条边
12   但系统状态没变
13   优先选"图效果大、系统效果小"的操作
```

BagAmmo (USENIX'23) — 相似度引导选择

3824字

Plain Text | 复制代码

- 1 安卓怎么做：
- 2 用opcode n-gram相似度选donor
- 3 选和恶意app代码风格最接近的良性app
- 4 注入后不突兀
- 5
- 6 参考什么：
- 7 选择注入内容时考虑和目标环境的匹配度
- 8
- 9 溯源图怎么用：
- 10 选良性行为时，选和当前环境正常行为分布最接近的
- 11 用操作类型的频率分布做匹配
- 12 ProvNinja的regularity score是一种实现
- 13 可以进一步用整体分布距离（KL散度等）衡量

EvadeDroid (C&S'24) — 作用域隔离

Plain Text | 复制代码

- 1 安卓怎么做：
- 2 移植Android组件（Activity/Service/Receiver）
- 3 组件之间天然隔离，不共享状态
- 4 新组件不影响恶意组件
- 5
- 6 参考什么：
- 7 让注入的内容和攻击内容的作用域不重叠
- 8
- 9 溯源图怎么用：
- 10 让良性操作在不同的作用域执行
- 11 不同namespace、不同pod、不同用户、不同文件目录
- 12 良性操作的影响范围和攻击依赖的资源尽量不重叠
- 13 这是 $S_{effect} \cap S_{dep} = \emptyset$ 的一种实现手段

HRAT (CCS'21) — 操作效果可预测

Plain Text | 复制代码

- 1 安卓怎么做：
- 2 定义4种原子操作
- 3 每个操作对FCG的影响是确定的
- 4 insert node → 精确多一个节点，不多不少
- 5 双重验证：静态检查FCG + 动态运行对比UI
- 6
- 7 参考什么：
- 8 每个操作的效果必须在执行前就能精确预测
- 9
- 10 溯源图怎么用：
- 11 建立每个良性操作到provenance graph的确定性映射
- 12 kubectl get pods → 产生process→file(read)边
- 13 curl http://... → 产生process→socket(connect)边
- 14 效果不可预测的操作不放进候选池
- 15 这是构建 S_effect 的基础

MalGuise (USENIX'24) — 安全空间前置

Plain Text | 复制代码

- 1 PE怎么做：
- 2 先定义安全变换空间 (semantic NOP + call-based redividing)
- 3 空间里所有变换都保证不破坏语义
- 4 搜索只在安全空间内进行
- 5 搜索算法不需要考虑"会不会破坏功能"
- 6
- 7 参考什么：
- 8 安全性保证和搜索优化彻底分离
- 9 先保证安全，再优化效果
- 10
- 11 溯源图怎么用：
- 12 先计算安全池 $B_safe = \{b \mid S_effect(b) \cap S_dep(A) = \emptyset\}$
- 13 然后在B_safe内用UCB/MCTS选择最优组合
- 14 选择算法只管evasion效果，不管会不会破坏攻击
- 15 安全性在搜索之前就已经保证了
- 16
- 17 和MalGuise的区别：
- 18 MalGuise的安全空间是静态的 (CPU语义决定，永远不变)
- 19 你的安全空间是动态的 (随攻击进度和环境状态变化)

总结

▼

Plain Text | 复制代码

1	论文	参考的思想	溯源图中怎么用
2			
3	BagAmmo	图效果大、系统效果小	优先选read-only/低副作用操作
4	BagAmmo	相似度引导选择	频率分布匹配选择良性行为
5	EvadeDroid	作用域隔离	不同namespace/pod/用户隔离作用域
6	HRAT	操作效果可预测	建立操作→graph的确定性映射
7	MalGuise	安全空间前置	先算B_safe再在其中搜索

五个参考点，解决不同层面的问题：

▼

Plain Text | 复制代码

1	选什么操作：BagAmmo（低副作用）+ BagAmmo（相似度）		
2	在哪执行：EvadeDroid（隔离）		
3	能不能预测：HRAT（确定性映射）		
4	怎么组织：MalGuise（安全空间前置）		

你要证明：现有方法搬到真实攻击场景，攻击会坏。

质疑二：隐蔽性

现有对抗样本方法只关注骗过ML检测器。但evasion过程本身（大量query、行为模式异常）在rule-based检测面前是裸奔的。

你要证明：现有方法的evasion行为，rule-based检测就能抓住。

隐蔽性方面，能参考的论文

AdvDroidZero (CCS'23) — 经验指导减少盲目尝试

Plain Text | 复制代码

- 1 安卓怎么做：
- 2 UCB公式平衡探索和利用
- 3 历史上有有效的perturbation优先选
- 4 语义相似的perturbation共享经验，不用每个都试
- 5
- 6 参考什么：
- 7 用经验指导选择，避免大量无效尝试
- 8
- 9 溯源图怎么用：
- 10 在安全池内用UCB选良性行为
- 11 某类操作有效降低了检测分数 → 同类操作优先选
- 12 减少总操作数 → 行为痕迹少 → 更隐蔽

MalGuise (USENIX'24) — 小搜索空间 + MCTS预测

Plain Text | 复制代码

- 1 PE怎么做：
- 2 只有两类变换，搜索空间本身就小
- 3 MCTS的simulation阶段用启发式估计，不用每次都query真实检测器
- 4 20次query达到97% ASR
- 5
- 6 参考什么：
- 7 搜索空间越小、预测能力越强，需要的实际操作越少
- 8
- 9 溯源图怎么用：
- 10 安全池过滤后搜索空间已经缩小
- 11 对候选操作预估evasion效果（比如用regularity score近似）
- 12 只对最有希望的操作才真正执行
- 13 减少不必要的操作 → 行为痕迹少

EvadeDroid (C&S'24) — 精准选择，一次到位

Plain Text | 复制代码

- 1 安卓怎么做：
- 2 用opcode相似度选最匹配的donor
- 3 第一次移植就大概率成功
- 4 1-9次query
- 5
- 6 参考什么：
- 7 选择越精准，需要尝试的次数越少
- 8
- 9 溯源图怎么用：
- 10 选良性行为时，用频率分布匹配度排序
- 11 最匹配正常环境的操作排在最前面
- 12 减少试错 → 总操作数少 → 更隐蔽

HagDe (USENIX'25) — 避免决策边界不稳定区域

Plain Text | 复制代码

- 1 安卓怎么做（防御侧）：
- 2 对抗样本在决策边界附近，微小perturbation就导致loss剧烈变化
- 3 利用这个不稳定性检测对抗样本
- 4
- 5 参考什么：
- 6 如果你的evasion让provenance graph落在决策边界附近，
- 7 容易被这类防御检测到
- 8
- 9 溯源图怎么用：
- 10 良性行为的插入要gradual、consistent
- 11 不要一次大幅改变行为分布
- 12 均匀穿插，让行为分布平滑偏移
- 13 避免让graph特征落在不稳定区域

BagAmmo (USENIX'23) — 相似度引导匹配分布



Plain Text | 复制代码

- 1 安卓怎么做：
- 2 选和恶意app opcode分布最接近的donor
- 3 注入后整体特征分布变化小
- 4
- 5 参考什么：
- 6 注入的内容要和目标环境的正常分布匹配
- 7
- 8 溯源图怎么用：
- 9 良性行为的类型分布要匹配正常环境
- 10 不能只选evasion效果好的，还要选分布匹配度高的
- 11 用KL散度等度量整体分布偏离
- 12 分布偏离小 → rule-based检测不到频率异常

总结

五个参考点，解决隐蔽性的不同层面：



Plain Text | 复制代码

- 1 减少操作数：AdvDroidZero (UCB) + MalGuise (小空间) + EvadeDroid (精准选择)
- 2 分布匹配： BagAmmo (相似度引导)
- 3 避免被检测： HagDe (平滑偏移)

完整实验设计

实验一：有效性——真实K8s场景

环境：本地K8s集群 (minikube/kind)

攻击链：2-3个典型攻击场景

▼ Plain Text | 复制代码

```
1  攻击1：容器逃逸
2    1. cat /var/run/secrets/kubernetes.io/serviceaccount/token
3    2. curl https://kubernetes.default/api/v1/pods
4    3. kubectl apply -f privileged-pod.yaml
5    4. mount /host
6    5. cat /host/etc/shadow
7
8  攻击2：横向移动
9    1. 扫描内网服务（curl各个service IP）
10   2. 读取configmap中的数据库凭据
11   3. 连接数据库
12   4. 导出数据
13   5. 外传
```

对比方法：

▼ Plain Text | 复制代码

```
1  方法A：原始攻击（不加良性行为）
2  方法B：随机插入良性行为（模拟现有方法）
3  方法C：ProvNinja风格插入（按regularity选gadget，不做安全过滤）
4  方法D：你的方法（安全过滤后插入）
```

方法B/C的良性行为池（20–30个常见操作）：

▼ Plain Text | 复制代码

```
1  只读操作：
2    kubectl get pods
3    kubectl get nodes
4    kubectl get services
5    kubectl get configmaps
6    kubectl describe pod xxx
7    kubectl logs xxx
8
9  有写入副作用的操作：
10   kubectl delete pod xxx
11   kubectl apply -f network-policy.yaml
12   kubectl scale deployment xxx --replicas=0
13   kubectl cordon node xxx
14   kubectl taint node xxx key=value:NoSchedule
15   kubectl create configmap xxx
16   kubectl set env deployment xxx KEY=VALUE
```

每次实验：从池中随机选5–10个良性操作，插入攻击步骤之间

度量指标：

▼ Plain Text 复制代码

```
1  指标1：攻击成功率（重复100次）
2  指标2：失败原因分类
3      - 哪个良性操作导致了失败
4      - 破坏了什么攻击依赖（网络连通性/Pod状态/权限/凭据）
```

预期结果：

▼ Plain Text 复制代码

```
1  方法      攻击成功率      原因
2  _____
3  A 原始攻击    100%          -
4  B 随机插入    50-70%        delete/scale/cordon等破坏了攻击依赖
5  C ProvNinja风格 60-80%      regularity高不代表对攻击安全
6  D 你的方法    ~100%         S_effect n S_dep = ∅ 过滤掉了危险操作
```

关键展示：

▼ Plain Text 复制代码

```
1  列一张表，每个良性操作标注：
2      操作      regularity      S_effect      是否和S_dep冲突      是否
3      kubectl get pods    高      ∅      否      否
4      kubectl delete pod  高      {Pod状态}      是      是
5      kubectl apply np    中      {NetworkPolicy,网络}      是      是
6      kubectl get nodes   高      ∅      否      否
7      kubectl scale        中      {Deployment,Pod数量}      是      是
8
9  结论：regularity高 ≠ 对攻击安全。ProvNinja只看regularity，不看S_effect。
```

实验二：隐蔽性——DARPA数据集

数据：DARPA TC数据集（ProvNinja用的同一个）

步骤：

▼ Plain Text 复制代码

```
1  步骤1：统计正常行为的频率baseline
2    从DARPA数据集的良性数据中统计：
3    - 每个程序每小时平均产生多少条边
4    - 每种操作类型（read/write/execute/connect）的频率分布
5    - 每个程序的操作类型分布
6    - 边数的时间序列（每分钟/每小时的边数变化）
7
8  步骤2：拿ProvNinja的evasion结果
9    ProvNinja对Enterprise APT加完gadget+camouflage后的provenance graph
10   ProvNinja对Supply Chain APT加完gadget+camouflage后的provenance graph
11
12 步骤3：统计ProvNinja evasion后的行为频率
13    同样统计：
14    - 每个程序的边数
15    - 操作类型频率分布
16    - 边数时间序列
```

检测规则（极简rule-based）：

▼ Plain Text 复制代码

```
1  规则1：边数异常
2    某程序的边数 > 正常均值 + 3σ → 告警
3
4  规则2：操作类型分布异常
5    某程序的操作类型分布和baseline的KL散度 > 阈值 → 告警
6
7  规则3：时间窗口内突增
8    某时间窗口（比如5分钟）内新增边数 > 正常窗口均值 + 3σ → 告警
```

对比方法：

▼ Plain Text 复制代码

```
1  方法A：原始攻击（不加camouflage）
2  方法B：ProvNinja的evasion结果
3  方法C：你的方法（频率匹配+均匀穿插）
```

度量指标：

▼ Plain Text 复制代码

```
1  指标1：ML检测器的检测率（用ProvNinja论文中的4个检测器）
2  指标2：频率规则的告警数量
3  指标3：行为分布距离 D_KL(P_attack || P_normal)
4  指标4：边数增长量（ProvNinja加了多少额外的边）
```

预期结果：

3824字

▼

Plain Text | 复制代码

1	ML检测率	频率规则告警	D_KL	额外边数	
2					
3	A 原始攻击	高	少（只有攻击本身）	低	0
4	B ProvNinja	低	多（camouflage触发）	高	大量
5	C 你的方法	低	少	低	适量

关键展示：

▼

Plain Text | 复制代码

1	图1：时间序列对比
2	x轴：时间
3	y轴：每分钟新增边数
4	
5	三条线：
6	- 正常环境baseline（平稳）
7	- ProvNinja evasion（某段时间突然暴增）
8	- 你的方法（和baseline接近）
9	
10	图2：操作类型分布对比
11	柱状图：read/write/execute/connect的比例
12	
13	三组柱子：
14	- 正常环境
15	- ProvNinja（某些类型比例明显偏高）
16	- 你的方法（分布接近正常）
17	
18	图3：检测结果汇总表
19	
20	
21	
22	

	ML检测器	规则1	规则2	规则3	
原始攻击	✗检测到	✓正常	✓正常	✓正常	
ProvNinja	✓逃过	✗异常	✗异常	✗异常	← 骗了ML但骗不了频率
你的方法	✓逃过	✓正常	✓正常	✓正常	← 两层都过

两个实验的关系

- 1 实验一（真实K8s）：
 - 2 证明现有方法加良性行为 → 攻击坏了
 - 3 你的安全过滤 → 攻击不坏
 - 4
- 5 实验二（DARPA数据集）：
 - 6 证明现有方法的evasion → 简单频率规则就能抓
 - 7 你的频率匹配 → 频率规则也抓不到
 - 8
- 9 合在一起：
 - 10 现有方法：攻击坏了 + evasion暴露了 = 两头都失败
 - 11 你的方法：攻击没坏 + evasion隐蔽 = 两头都成功

定义良性行为会干扰隐蔽攻击

检测器黑盒构建

研究对象看人下菜碟